

Yesterday 3:49 PM

Dependency Injection / IoC

What is IoC?

What is Dependency Injection?

Why do we need Dependency Injection?

Constructor injection vs setter injection vs field injection.

Why is constructor injection preferred?

What is the Spring IoC container?

BeanFactory vs ApplicationContext.

How does Spring create a bean?

How does Spring manage dependencies between beans?

Show more ▾

Absolutely. For a 5-year Java/Spring Boot interview, don't just memorize definitions. Interviewers usually go one level deeper: **why, how Spring does it internally, what happens in edge cases, and which approach you'd choose in production.**

Dependency Injection / IoC — Interview Guide

1. What is IoC?

IoC (Inversion of Control) means that the responsibility of creating objects and managing their dependencies is transferred from the application code to a framework/container.

Without IoC

</> Java



```
public class OrderService {  
  
    private PaymentService paymentService = new PaymentService();  
  
}
```

OrderService directly creates PaymentService .

So:

```

OrderService
  |
  └─ creates PaymentService
  
```

This creates tight coupling.

With Spring IoC

</> **Java**

```

@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
  
```

Spring creates both objects and provides `PaymentService` to `OrderService`.

```

Spring IoC Container
  |
  +---- PaymentService
  |
  +---- OrderService
         |
         └─ PaymentService
  
```

Interview answer

IoC means transferring control of object creation, configuration, and dependency management from application code to a container such as the Spring IoC container.

2. What is Dependency Injection?

Dependency Injection is one way of implementing IoC.

Suppose:

</> Java



```
class OrderService {  
    private PaymentService paymentService;  
}
```

OrderService depends on PaymentService .

Instead of creating it:

</> Java



```
PaymentService paymentService = new PaymentService();
```

we let Spring provide it:

</> Java



```
@Service  
class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

So:

Dependency



PaymentService

Injected into



OrderService



Important distinction

IoC



Broad principle

Dependency Injection





Technique used to achieve IoC

A strong interview answer:

IoC is the principle where control of object creation and dependency management is transferred to the container, while Dependency Injection is the mechanism through which the container supplies required dependencies to objects.

3. Why do we need Dependency Injection?

The biggest reason is **loose coupling**.

Suppose:

</> Java



```
class OrderService {  
  
    private PaymentService paymentService = new PaymentService();  
  
}
```

Now `OrderService` is tightly coupled to a concrete implementation.

If we change:

PaymentService



UPIPaymentService



we have to modify `OrderService`.

Instead:

</> Java



```
interface PaymentService {  
    void pay();  
}
```

</> Java



```
@Service
class UPIPaymentService implements PaymentService {
    public void pay() {}
}
```

</> Java



```
@Service
class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now `OrderService` depends on an abstraction.

Benefits

- Loose coupling
- Easier testing
- Easier replacement of implementations
- Better maintainability
- Centralized object configuration
- Supports SOLID principles
- Easier mocking

For example:

</> Java



```
PaymentService mockPaymentService
```

can be injected during testing.

4. Constructor Injection vs Setter Injection vs Field Injection

Constructor Injection

</> Java



```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Dependency is supplied during object creation.

Setter Injection

</> Java



```
@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Dependency is provided after object creation.

Useful when dependency is **optional** or **can be changed later**.

Field Injection

</> Java



```
@Service
public class OrderService {
```

```
@Autowired  
private PaymentService paymentService;  
}
```

Spring uses reflection to inject the dependency into the private field.

5. Why is Constructor Injection preferred?

This is a very common 5-year interview question.

Reason 1 — Dependency is mandatory

```
</> Java  
  
public OrderService(PaymentService paymentService)
```



You cannot construct the service without its required dependency.

Reason 2 — Supports immutability

```
</> Java  
  
private final PaymentService paymentService;
```



Once assigned, it cannot be reassigned.

Reason 3 — Easier unit testing

You can simply do:

```
</> Java  
  
PaymentService mock = mock(PaymentService.class);  
  
OrderService service = new OrderService(mock);
```



No Spring context required.

Reason 4 — Makes dependencies explicit

Look at:

</> Java



```
public OrderService(  
    PaymentService paymentService,  
    InventoryService inventoryService,  
    OrderRepository repository) {  
}
```

You immediately know what `OrderService` requires.

With field injection, dependencies are hidden inside the class.

Reason 5 — Fail fast

If a mandatory dependency is missing, application startup fails instead of discovering the problem later.

Interview answer

Constructor injection is preferred because it makes dependencies explicit and mandatory, supports final fields and immutability, improves testability, and allows Spring to fail fast when a required dependency cannot be resolved.

6. What is the Spring IoC Container?

The Spring IoC container is responsible for:

Create beans



Configure beans



Resolve dependencies



Inject dependencies



Manage bean lifecycle



Destroy beans when appropriate



Common containers:

BeanFactory
ApplicationContext

In Spring Boot applications, you normally interact with:

</> Java
ApplicationContext

rather than directly working with BeanFactory .

7. BeanFactory vs ApplicationContext

BeanFactory	ApplicationContext
Basic IoC container	More feature-rich container
Lazy bean creation generally	Supports eager singleton initialization by default
Basic dependency injection	DI + events + i18n + resource loading
Lightweight	More commonly used
Lower-level	Higher-level

Interview answer

BeanFactory provides the basic IoC functionality, while ApplicationContext extends it with additional enterprise features such as application events, internationalization, resource loading, and easier integration with Spring infrastructure.

In Spring Boot, ApplicationContext is the normal choice.

8. How does Spring create a bean?

Suppose:

</> Java



```
@Service
public class PaymentService {
}
```

During startup, Spring scans configured packages.

It finds:

</> Java



```
@Service
```

and registers a **BeanDefinition**.

Conceptually:

```
@Component scanning
```

↓

```
Find @Service
```

↓

```
Create BeanDefinition
```

↓

```
Register in BeanFactory
```

↓

```
Instantiate bean
```

↓

```
Resolve dependencies
```

↓

```
Inject dependencies
```

↓

```
Initialize bean
```

↓

```
Bean ready
```



For example:

</> Java



```
@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
```

```

        this.paymentService = paymentService;
    }
}

```

Spring recognizes that:

```

OrderService
    ↓
requires PaymentService

```



So it finds the `PaymentService` bean and passes it into the constructor.

9. How does Spring manage dependencies between beans?

Suppose:

```

</> Java

@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

```



Spring maintains a registry of beans.

Conceptually:

```

Bean Container

PaymentService → object
OrderService   → object

```



When creating `OrderService`, Spring sees:

```

OrderService requires PaymentService

```



It resolves:

```
PaymentService
  ↓
inject
  ↓
OrderService
```



If `PaymentService` itself has dependencies, Spring resolves those recursively.

```
OrderService
  |
  ↓
PaymentService
  |
  ↓
PaymentRepository
```



Spring builds the dependency graph.

10. What happens when two beans have the same type?

This is extremely important.

Suppose:

```
<> Java

interface PaymentService {
    void pay();
}
```



You have:

```
<> Java

@Service
class UpiPaymentService implements PaymentService {
}
```



and:

</> **Java**

```
@Service
class CardPaymentService implements PaymentService {
}
```

Now:

</> **Java**

```
@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

PaymentService



```
|
+---- UpiPaymentService
|
+---- CardPaymentService
```

It cannot determine which one to inject.

You generally get:

NoUniqueBeanDefinitionException



11. @Autowired vs @Qualifier

@Autowired tells Spring:

Find a suitable bean for this dependency.

Example:

</> Java



```
@Autowired  
private PaymentService paymentService;
```

If there is only one implementation, it works.

But if there are multiple implementations, use:

</> Java



```
@Autowired  
@Qualifier("upiPaymentService")  
private PaymentService paymentService;
```

Or preferably constructor injection:

</> Java



```
public OrderService(  
    @Qualifier("upiPaymentService")  
    PaymentService paymentService) {  
  
    this.paymentService = paymentService;  
}
```

12. @Primary vs @Qualifier

Suppose:

</> Java



```
@Service  
@Primary  
class UpiPaymentService implements PaymentService {  
}
```

and:

</> Java



```
@Service  
class CardPaymentService implements PaymentService {
```

}

Now:

</> Java



```
OrderService(PaymentService paymentService)
```

will receive:

UpiPaymentService



because it is marked @Primary .

What if I want CardPaymentService?

Use:

</> Java



```
OrderService(  
    @Qualifier("cardPaymentService")  
    PaymentService paymentService  
)
```

Key difference

@Primary :

Default choice when multiple candidates exist.

@Qualifier :

Explicitly select this particular bean.

Priority

Think:

```
@Qualifier  
  ↓  
explicit choice  
  
@Primary
```



↓
default choice

If a qualifier identifies a bean, it takes precedence over simply choosing the primary candidate.

13. Can Spring inject private fields?

Yes.

Example:

```
</> Java   
  
@Service  
public class OrderService {  
  
    @Autowired  
    private PaymentService paymentService;  
  
}
```

Spring can perform field injection using reflection.

It doesn't require the field to be public.

But should you use it?

Generally, no for mandatory dependencies.

Prefer:

```
</> Java   
  
private final PaymentService paymentService;  
  
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

Interview answer

Yes, Spring can inject private fields through reflection, but constructor injection is generally preferred because it makes dependencies explicit, supports immutability, and

improves testability.

14. Can Spring inject a bean through a constructor?

Yes. This is the preferred approach.

</> Java



```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

In modern Spring, if the class has a **single constructor**, you generally don't need `@Autowired`.

So this is enough:

</> Java



```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

You don't need:

</> Java



```
@Autowired
public OrderService(...)
```

for the single-constructor case.

15. What happens if a dependency is missing?

Suppose:

</> Java



```
@Service
class OrderService {

    OrderService(PaymentService paymentService) {
    }
}
```

But Spring cannot find a bean of type:

PaymentService



Then application startup normally fails with an error such as:

NoSuchBeanDefinitionException



or, depending on the resolution path/context, a bean-creation/dependency-resolution exception wrapping that problem.

The important interview point:

```
Required dependency missing
      ↓
Spring cannot create dependent bean
      ↓
Application context fails to start
```



This is one reason constructor injection is useful: **fail fast**.

16. What happens if multiple beans match the same interface?

Example:

</> Java



```
interface NotificationService {
}
```

</> Java



```
@Service
class EmailNotificationService implements NotificationService {
}
```

</> Java



```
@Service
class SmsNotificationService implements NotificationService {
}
```

Then:

</> Java



```
NotificationService service
```

has two candidates.

Spring cannot choose automatically.

You need one of:

@Qualifier

</> Java



```
public OrderService(
    @Qualifier("emailNotificationService")
    NotificationService service) {
}
```

@Primary

</> Java



```
@Service
@Primary
class EmailNotificationService
    implements NotificationService {
}
```

Then:

</> Java



```
NotificationService service
```

will use the primary bean.

17. How would you conditionally create a bean?

This is a good 5-year Spring Boot question.

Spring Boot provides several mechanisms.

`@ConditionalOnProperty`

Very common.

</> Java



```
@Configuration
public class PaymentConfig {

    @Bean
    @ConditionalOnProperty(
        name = "payment.enabled",
        havingValue = "true"
    )
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Configuration:

</> properties



```
payment.enabled=true
```

Then bean is created.

If:

</> properties



```
payment.enabled=false
```

the bean isn't created.

@Profile

You can create beans for specific environments.

</> **Java**



```
@Bean
@Profile("dev")
public PaymentService devPaymentService() {
    return new MockPaymentService();
}
```

And:

</> **Java**



```
@Bean
@Profile("prod")
public PaymentService prodPaymentService() {
    return new RealPaymentService();
}
```

So:

dev → MockPaymentService



prod → RealPaymentService

@ConditionalOnMissingBean

Very common when building reusable Spring Boot configuration.

</> **Java**



```
@Bean
@ConditionalOnMissingBean
public PaymentService paymentService() {
    return new DefaultPaymentService();
}
```

Meaning:

Create this bean only if another bean of this type hasn't already been provided.

This is heavily used in Spring Boot auto-configuration.

5-Year Interview Scenario

Interviewer:

We have three implementations of `PaymentService` : UPI, Card, and Wallet. The application should use UPI by default, but certain services should use Card. How would you design it?

Good answer:

```
</> Java   
  
@Service  
@Primary  
class UpiPaymentService implements PaymentService {  
}
```

Then the normal injection:

```
</> Java   
  
public PaymentProcessor(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

gets UPI.

For a service requiring Card:

```
</> Java   
  
public CardOrderService(  
    @Qualifier("cardPaymentService")  
    PaymentService paymentService) {  
  
    this.paymentService = paymentService;  
}
```

So:

Default

↓

@Primary

↓

UPI

Specific requirement

↓

@Qualifier

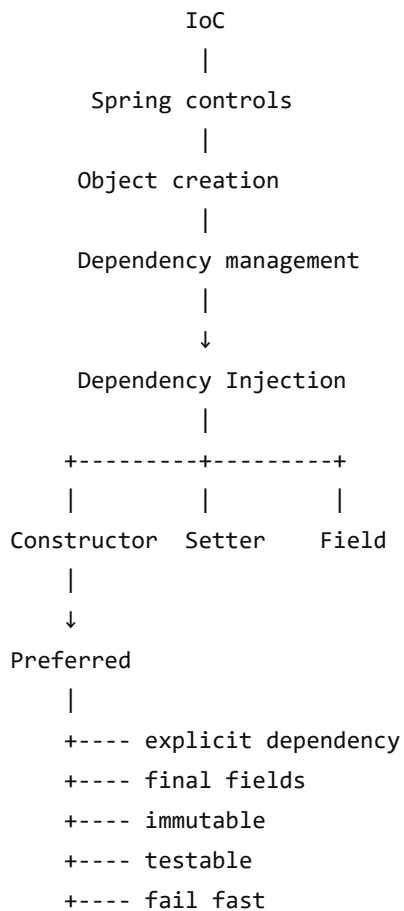
↓

Card



The Interview Mental Model

Remember this entire topic as:



And for multiple beans:

One interface

|

+----- Bean A

|

+----- Bean B

|

↓

Multiple candidates

|

+----- @Qualifier → explicit bean

|

+----- @Primary → default bean



And conditional beans:

Should bean exist?

|

+----- @Profile

|

+----- @ConditionalOnProperty

|

+----- @ConditionalOnMissingBean



★ Questions you should be ready for next

For a 5-year Spring Boot interview, the interviewer can easily continue with:

1. What exactly happens during Spring bean lifecycle?
2. What is a BeanDefinition?
3. What is component scanning?
4. @Component VS @Service VS @Repository ?
5. What happens internally when @Autowired is used?
6. How does Spring resolve constructor dependencies?
7. What is circular dependency?
8. Can constructor injection create circular dependency?
9. What is @Lazy and when would you use it?
10. Singleton vs prototype bean scope?
11. What happens when a singleton bean depends on a prototype bean?
12. How does Spring handle circular dependencies?
13. @Bean VS @Component ?
14. @Configuration and proxy behavior?
15. What is Spring BeanPostProcessor?

16. What is `BeanFactoryPostProcessor` ?
17. What is `ApplicationContext` startup sequence?
18. How does Spring Boot auto-configuration work?
19. How does `@ConditionalOnMissingBean` work internally?
20. What happens if two `@Configuration` classes define beans with the same name?



1. Monolith vs Microservices

- **Monolith** — The application is built and deployed as a single unit.
- **Microservices** — The application is split into independently deployable services, each owning a specific business capability.

Monolith:



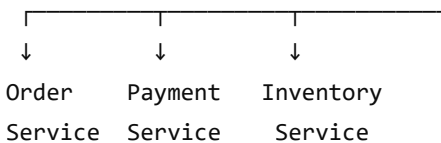
Client → Application → Database

Microservices:

Client



API Gateway



2. When should you choose Microservices?

Choose microservices when the system has sufficiently large/complex domains, requires independent scaling or deployment, and the organization can handle the operational complexity.

Good reasons:

- Independent scaling
- Independent deployments
- Clear business boundaries
- Different teams owning different services
- Fault isolation
- Different technology requirements

Don't choose microservices simply because they are popular; a small application may be better as a monolith.

3. Advantages of Microservices

- **Independent deployment** — Services can be deployed independently.
- **Independent scaling** — Scale only the service that needs more capacity.
- **Fault isolation** — Failure in one service can potentially be contained.
- **Team autonomy** — Different teams can own different services.
- **Clear business boundaries** — Services can represent business capabilities.

4. Disadvantages of Microservices

- Distributed-system complexity
- Network failures
- Distributed transactions
- More deployment/monitoring infrastructure
- Service-to-service latency
- Data consistency challenges
- More difficult debugging
- Distributed tracing requirements
- More operational overhead

5. Service Discovery

Service discovery allows services to dynamically find the network location of other service instances instead of hardcoding their addresses.

Example:

```
Order Service
  ↓
Service Discovery
  ↓
Payment Service
```



↓
10.0.2.15:8080

Common approaches include:

Service Registry
Eureka
Consul
Kubernetes Service/DNS



6. API Gateway

API Gateway is an entry point for client requests that can perform routing, authentication, rate limiting, TLS termination, and other cross-cutting concerns.

Client
↓
API Gateway
├─> Order Service
├─> Payment Service
└─> User Service



It can prevent clients from needing to know every internal service address.

7. Load Balancing

Load balancing distributes incoming requests across multiple healthy instances of a service.

Load Balancer
/ | \
↓ ↓ ↓
Instance1 Instance2 Instance3



Benefits:

- Better availability
- Better resource utilization
- Horizontal scaling
- Failure handling

8. Client-Side vs Server-Side Load Balancing

Client-side

The client discovers service instances and selects which instance to call.

```
Order Service
  ↓
Service Registry
  ↓
Instance list
  ↓
Client selects instance
```



Server-side

The client sends the request to a load balancer, which selects the service instance.

```
Order Service
  ↓
Load Balancer
  ↓
Instance 1 / 2 / 3
```



Difference

Client-side → Client chooses destination



Server-side → Load balancer chooses destination

9. OpenFeign

OpenFeign is a declarative HTTP client that allows you to define service-to-service HTTP calls using Java interfaces.

Example:

```
</> Java
```



```
@FeignClient(name = "payment-service")
public interface PaymentClient {

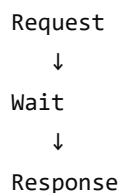
    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Instead of manually constructing HTTP requests, you define the API contract and the framework creates the client implementation.

10. RestClient vs WebClient

RestClient

RestClient is Spring's synchronous HTTP client API for making blocking HTTP calls.



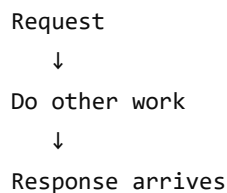
```
graph TD
    Request --> Wait
    Wait --> Response
```

Request
↓
Wait
↓
Response



WebClient

WebClient is Spring's reactive, non-blocking HTTP client designed for asynchronous/reactive communication.



```
graph TD
    Request --> DoOtherWork[Do other work]
    DoOtherWork --> ResponseArrives[Response arrives]
```

Request
↓
Do other work
↓
Response arrives



For a traditional Spring MVC application, **RestClient** is often simpler; **WebClient** is particularly useful in reactive applications or when non-blocking I/O is important.

11. Synchronous vs Asynchronous Communication

Synchronous

The caller waits for the downstream service response before continuing.

```

Order
↓
Payment
↓
Response
↓
Order continues

```



Example:

```

REST
Feign
RestClient

```



Asynchronous

The caller sends a message/event and does not need to wait for the consumer to finish processing it.

```

Order Service
↓
Kafka
↓
Payment Service

```



Key difference

```

Synchronous → Request/response
Asynchronous → Message/event based

```



12. Circuit Breaker

A circuit breaker prevents repeated calls to a failing downstream service and gives that service time to recover.

States:

```

CLOSED
↓ failures exceed threshold
OPEN
↓ after wait period

```



```
HALF_OPEN
↓
Success → CLOSED
Failure → OPEN
```

Example:

```
Payment Service is down
↓
Order
↓
Circuit Breaker
↓
Don't repeatedly call Payment
↓
Fallback / error
```



Common Spring implementation:

```
Resilience4j
```



13. Retry

Retry automatically repeats a failed operation when the failure is considered temporary.

Example:

```
Request
↓
Payment Service
↓
Timeout
↓
Retry
↓
Payment Service
↓
Success
```



Use retry carefully.

Good candidates:

Temporary network failure
Transient service unavailability
Temporary timeout



Bad candidates:

Invalid request
Authentication failure
Permanent business validation failure



Retries should normally use a limit and backoff, often exponential backoff with jitter.

14. Timeout

A timeout defines the maximum amount of time a service will wait for a downstream operation before failing it.

Without a timeout:

Service A
↓
Service B hangs
↓
Thread waits indefinitely
↓
Resources exhausted



With timeout:

Service A
↓
Service B
↓
Timeout
↓
Fail fast / fallback



Timeouts are essential for preventing slow downstream services from consuming resources indefinitely.

15. Bulkhead

Bulkhead isolates resources so that failure or overload in one operation does not consume all resources of the application.

Named after ship bulkheads: one damaged section should not sink the entire ship.

Example:

```
Application
├─ Payment Pool
├─ Inventory Pool
└─ Notification Pool
```



If Payment becomes slow:

```
Payment Pool → Exhausted

Inventory Pool → Still available
```



This prevents one dependency from taking down unrelated functionality.

16. Rate Limiting

Rate limiting restricts how many requests a client or service can make within a specified period.

Example:

```
Client
↓
Rate Limiter
↓
100 requests/minute
```



Request 101:

```
Rate limit exceeded
↓
HTTP 429
```



Common algorithms include:

Token Bucket
Leaky Bucket
Fixed Window
Sliding Window



17. Distributed Tracing

Distributed tracing tracks a request across multiple services using a trace ID and spans.

Client
↓
Gateway
↓
Order
↓
Payment
↓
Inventory



Conceptually:

Trace ID: ABC123



Gateway	— Span 1
Order	— Span 2
Payment	— Span 3
Inventory	— Span 4

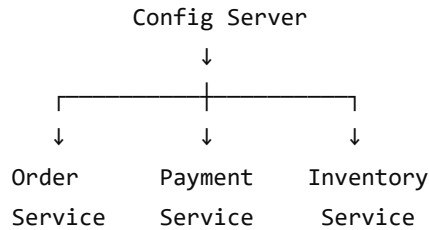
This helps identify:

- Which service is slow
- Where an exception occurred
- End-to-end latency
- Downstream dependency problems

OpenTelemetry is commonly used for instrumentation and context propagation.

18. Centralized Configuration

Centralized configuration stores application configuration in a central system so services don't need to maintain configuration independently in every deployment.



Examples:

Spring Cloud Config
Consul
Kubernetes ConfigMap/Secret
Cloud-native configuration services



Benefits:

- Centralized management
- Environment-specific configuration
- Easier configuration changes
- Reduced duplication

Secrets should be handled through appropriate secret-management systems rather than ordinary configuration files.

19. Distributed Transactions

A distributed transaction is a business operation that spans multiple services/databases and therefore cannot usually be handled by a single local database transaction.

Example:

Order Service → Order DB
↓
Payment Service → Payment DB
↓
Inventory Service → Inventory DB



You cannot simply use one normal local transaction across all these independent databases.

Common approaches include:

Saga
Outbox pattern
Compensating transactions



20. Saga Pattern

Saga breaks a distributed business transaction into a sequence of local transactions, with compensating actions used when later steps fail.

Example:

Create Order
↓
Reserve Inventory
↓
Process Payment
↓
Order Confirmed



If payment fails:

Create Order
↓
Reserve Inventory
↓
Payment FAILED
↓
Release Inventory
↓
Cancel Order



The compensating actions undo the business effects of earlier successful steps.

Two common Saga styles:

Choreography → Services react to events

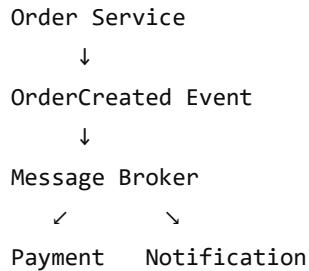


Orchestration → Central Saga orchestrator coordinates steps

21. Event-Driven Architecture

Event-driven architecture uses events to communicate that something happened, allowing other components to react asynchronously.

Example:



The producer doesn't necessarily need to know which consumers process the event.

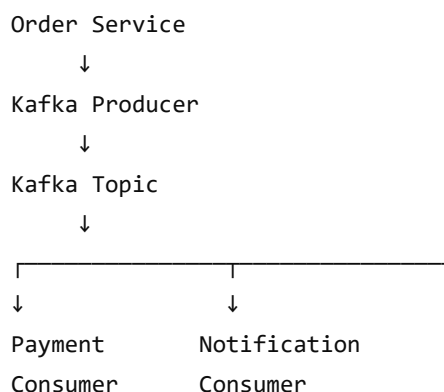
Benefits:

- Loose coupling
- Asynchronous processing
- Scalability
- Easy addition of new consumers

22. Kafka Integration

Kafka is a distributed event-streaming platform commonly used to publish, store, and consume events between services.

Typical architecture:



Example event:

```
OrderCreated
{
  "orderId": 123,
  "customerId": 456
}
```



Important Kafka concepts:

```
Producer
Consumer
Topic
Partition
Offset
Consumer Group
Broker
```



23. Idempotency

An operation is idempotent when performing it multiple times produces the same intended business result as performing it once.

Example:

```
POST payment request
  ↓
Network timeout
  ↓
Client retries
  ↓
Same payment request arrives again
```



Without idempotency:

```
Payment 1
Payment 2 ❌
```



With an idempotency key:

```
Idempotency-Key: ABC123

First request:
ABC123 → Process payment
```

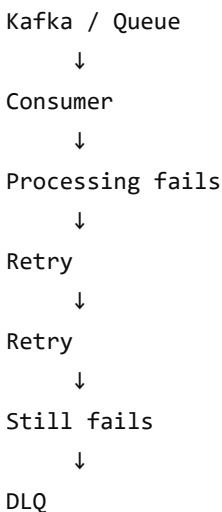


Retry:
ABC123 → Return existing result

This is especially important for payment/order APIs and message consumers.

24. Dead-Letter Queue

A Dead-Letter Queue (DLQ) stores messages that cannot be successfully processed after configured retry attempts or because they are otherwise considered unprocessable.



```
graph TD; A[Kafka / Queue] --> B[Consumer]; B --> C[Processing fails]; C --> D[Retry]; D --> E[Retry]; E --> F[Still fails]; F --> G[DLQ];
```

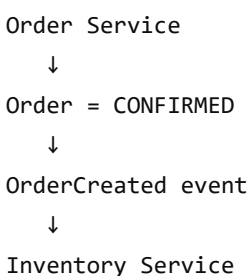
Kafka / Queue
↓
Consumer
↓
Processing fails
↓
Retry
↓
Retry
↓
Still fails
↓
DLQ

The DLQ allows engineers to inspect and reprocess problematic messages without continuously blocking normal processing.

25. Eventual Consistency

Eventual consistency means different services or replicas may temporarily have different data, but they converge to a consistent state after successful asynchronous processing.

Example:



```
graph TD; A[Order Service] --> B[Order = CONFIRMED]; B --> C[OrderCreated event]; C --> D[Inventory Service];
```

Order Service
↓
Order = CONFIRMED
↓
OrderCreated event
↓
Inventory Service

↓
Inventory updated shortly afterward

There can be a short period:

Order DB → CONFIRMED
Inventory DB → old state



Later:

Order DB → CONFIRMED
Inventory DB → updated



This is common in event-driven microservices because services often own separate databases and communicate asynchronously.

Important 5-Year Scenario

"Order → Payment → Inventory. Payment is slow. How would you design it?"

A strong answer would be:

"I would first define whether payment must be synchronous for the user flow. For synchronous calls, I would use a strict timeout, limited retry with exponential backoff and jitter for transient failures, and a circuit breaker to prevent cascading failures. I'd use bulkheads to isolate payment resources from other operations. If the business permits asynchronous processing, I'd publish an order/payment event through Kafka and use an event-driven Saga for the overall workflow. I'd make payment operations idempotent so retries don't cause duplicate payments, use a DLQ for messages that repeatedly fail, and use distributed tracing with correlation/trace IDs to monitor the complete flow."



What is API Gateway?

Why do we need an API Gateway?

Gateway vs Load Balancer.

What is service discovery?

Eureka vs Consul.

What happens when service discovery is unavailable?

How does OpenFeign work?

Feign timeout configuration.

Retry configuration.

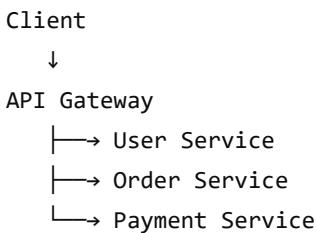
Circuit breaker with Resilience4j.

What is fallback?

Show more ▾

1. What is API Gateway?

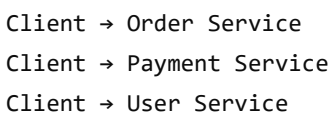
API Gateway is a single entry point for clients that routes requests to backend microservices and can handle cross-cutting concerns such as authentication, rate limiting, routing, and monitoring.



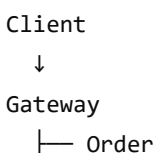
2. Why do we need an API Gateway?

We use an API Gateway to hide internal service topology from clients and centralize common concerns such as authentication, routing, rate limiting, CORS, logging, and request transformation.

Without gateway:



With gateway:



```
└─ Payment
└─ User
```

3. Gateway vs Load Balancer

API Gateway

API Gateway operates mainly at the application/API level and understands routes, APIs, authentication, and policies.

Example:

```
/api/orders/** → Order Service
/api/payments/** → Payment Service
```



Load Balancer

Load balancer distributes traffic across healthy instances of a service or application.

```
Order Service
  ↓
Load Balancer
  ✓ ↓ \
 01 02 03
```



Key difference

```
API Gateway
→ Which service should handle this request?
```



```
Load Balancer
→ Which healthy instance of that service should handle it?
```

They can be used together:

```
Client
  ↓
API Gateway
  ↓
Load Balancer
```



↓

Order-1 / Order-2 / Order-3

4. What is Service Discovery?

Service discovery allows services to dynamically find the network locations of other service instances.

Instead of hardcoding:

http://10.10.1.20:8080

📄

a service asks a discovery mechanism:

Where is payment-service?

📄

and receives available instances:

payment-service

- └ 10.10.1.20:8080
- └ 10.10.1.21:8080
- └ 10.10.1.22:8080

📄

5. Eureka vs Consul

	Eureka	Consul
Created by	Netflix	HashiCorp
Service discovery	Yes	Yes
Health checks	Yes	Yes
Key/value configuration	Limited compared with Consul	Yes
Multi-platform ecosystem	Primarily Java/Spring ecosystem	Broad

	Eureka	Consul
Common Spring integration	Spring Cloud Netflix	Spring Cloud Consul

Simple interview answer

Eureka is primarily focused on service discovery within the Netflix/Spring ecosystem, while Consul provides service discovery plus health checking, service networking features, and a distributed key-value store.

6. What happens when Service Discovery is unavailable?

It depends on the architecture and whether services have cached service information, but new discovery operations may fail and services may be unable to locate newly registered instances.

For example:

```
graph TD
    A[Order Service] --> B[Service Discovery ✗]
    B --> C[Cannot discover Payment]
```



If the client has cached previously discovered instances:

```
graph TD
    A[Service Discovery ✗] --> B[Cached instance list]
    B --> C[Continue calling known instances]
```



But this can become problematic because:

- New instances may not be discovered
- Removed/unhealthy instances may remain in stale cache
- Scaling information may become outdated
- New deployments may not be visible

Production design should avoid making service discovery a single point of failure and should use redundancy/caching appropriate to the discovery technology.

7. How does OpenFeign work?

OpenFeign provides a declarative HTTP client where you define an interface and Spring creates the client implementation that performs HTTP calls.

Example:

`</>` Java



```
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Then:

`</>` Java



```
Payment payment =
    paymentClient.getPayment(100);
```

Conceptually:

```
Order Service
  ↓
Feign Proxy
  ↓
Service Discovery
  ↓
Payment Instance
  ↓
HTTP
```



Feign handles the HTTP client plumbing so the application code can work with an interface.

8. Feign Timeout Configuration

Feign timeouts define how long the client waits for connection establishment and response processing before failing the call.

Conceptually:

`</>` **YAML**



```
spring:
  cloud:
    openfeign:
      client:
        config:
          payment-service:
            connectTimeout: 2000
            readTimeout: 3000
```

Meaning:

Connection timeout → 2 seconds
Read/response timeout → 3 seconds



The exact configuration properties can vary by Spring Cloud version, so the project's version-specific documentation should be followed.

9. Retry Configuration

Retry configuration determines whether and how failed calls are attempted again.

Example concept:

```
Request
↓
Payment
↓
Timeout
↓
Retry #1
↓
Timeout
↓
Retry #2
↓
Failure
```



A production retry policy should define:

Maximum attempts
Backoff
Jitter
Retryable exceptions/statuses
Overall timeout/deadline



Don't blindly retry every failure.

10. Circuit Breaker with Resilience4j

Resilience4j Circuit Breaker monitors failures and temporarily stops calls to an unhealthy dependency when failure thresholds are exceeded.

Example:

</> Java



```
@CircuitBreaker(  
    name = "paymentService",  
    fallbackMethod = "paymentFallback"  
)  
public Payment makePayment() {  
    return paymentClient.pay();  
}
```

Flow:

CLOSED
↓
Failures increase
↓
Threshold reached
↓
OPEN
↓
Calls fail fast
↓
After wait duration
↓
HALF_OPEN
↓
Test calls



11. What is Fallback?

Fallback is an alternative response or action executed when a protected operation fails or the circuit breaker is open.

Example:

`</>` Java

```
public Payment paymentFallback(  
    Throwable ex) {  
  
    return Payment.pending();  
}
```



Conceptually:

Payment Service



FAIL



Circuit Breaker



Fallback



Return alternative response



A fallback should be **business-safe**; it should not hide serious failures or incorrectly report that an operation succeeded.

12. Retry vs Circuit Breaker

Retry

Retry says: "The failure may be temporary, so try again."

Call → Failure → Retry → Retry → Success



Circuit Breaker

Circuit breaker says: "The dependency is currently unhealthy, so stop calling it for a while."

```
Failures
↓
Circuit opens
↓
Fail fast
```



They can work together

```
Request
↓
Circuit Breaker
↓
Retry
↓
Payment Service
```



But they must be configured carefully.

13. Why can retries sometimes make a production outage worse?

Retries can multiply traffic against an already overloaded or failing service, causing a retry storm and making the outage worse.

Suppose:

```
100 requests/sec
```



Each request retries twice:

```
100 original
+ 100 retry 1
+ 100 retry 2
= up to 300 attempts/sec
```



Now imagine 10 services doing similar retries:

```
Service A
↓
Service B
```



↓
Service C

The extra traffic can propagate through the system.

This can cause:

Service overloaded
↓
Requests become slow
↓
Timeouts
↓
Retries
↓
More requests
↓
More overload
↓
Cascading failure



Therefore use:

Limited retries
Exponential backoff
Jitter
Timeouts
Circuit breakers
Rate limits



14. What is Bulkhead Isolation?

Bulkhead isolation limits the resources available to one dependency or workload so that its failure cannot consume all application resources.

Example:

Application
├─ Payment calls → 20 threads
├─ Inventory calls → 20 threads
└─ Notification calls → 10 threads



If Payment becomes slow:

Payment pool → Exhausted
Inventory → Still has resources
Notification → Still has resources



Without isolation:

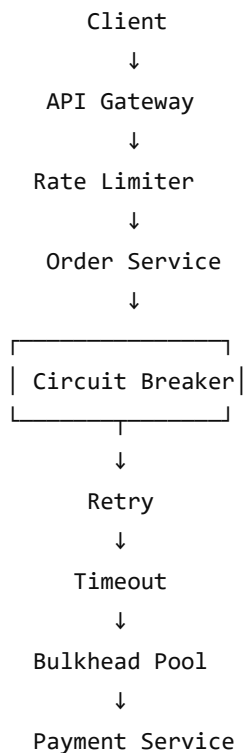
Payment calls consume all threads
↓
No threads left
↓
Entire application becomes unhealthy



15. How would you prevent cascading failures?

I would combine timeouts, limited retries with backoff/jitter, circuit breakers, bulkheads, rate limiting, bounded queues/thread pools, graceful degradation, and proper monitoring/tracing.

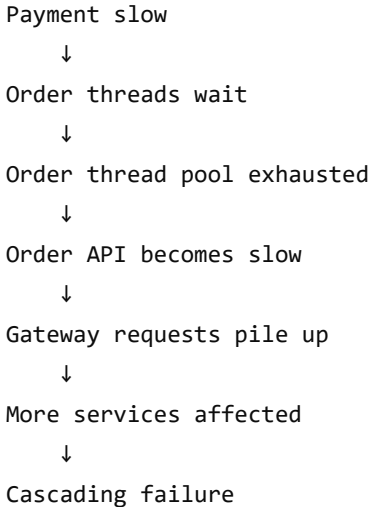
A typical design:



Example production failure

Suppose Payment Service becomes slow.

Without protection:

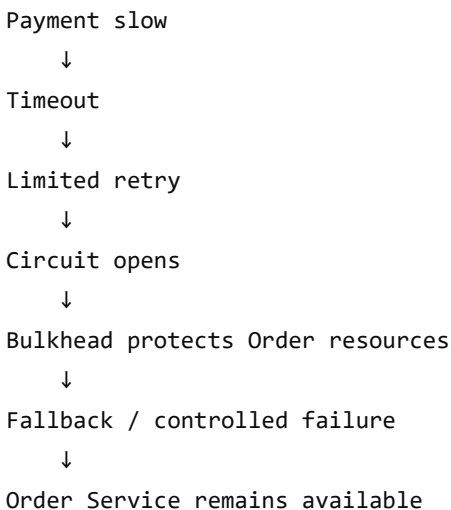


```
graph TD; A[Payment slow] --> B[Order threads wait]; B --> C[Order thread pool exhausted]; C --> D[Order API becomes slow]; D --> E[Gateway requests pile up]; E --> F[More services affected]; F --> G[Cascading failure];
```

Payment slow
↓
Order threads wait
↓
Order thread pool exhausted
↓
Order API becomes slow
↓
Gateway requests pile up
↓
More services affected
↓
Cascading failure



With resilience patterns:



```
graph TD; A[Payment slow] --> B[Timeout]; B --> C[Limited retry]; C --> D[Circuit opens]; D --> E[Bulkhead protects Order resources]; E --> F[Fallback / controlled failure]; F --> G[Order Service remains available];
```

Payment slow
↓
Timeout
↓
Limited retry
↓
Circuit opens
↓
Bulkhead protects Order resources
↓
Fallback / controlled failure
↓
Order Service remains available



Strong 5-year interview answer

"I would first set realistic timeouts so calls don't wait indefinitely. For transient failures I'd use limited retries with exponential backoff and jitter, but only for operations that are safe to retry. I'd put a circuit breaker around unstable dependencies so repeated failures don't continuously consume resources. I'd use bulkheads to isolate thread pools or concurrency for critical downstream services, rate limiting to control incoming load, and bounded queues to prevent unbounded resource consumption. Finally, I'd use distributed tracing, metrics, and alerts to detect latency, error rates, and dependency failures early."



Unit test vs integration test.

JUnit 5.

Mockito.

@Mock.

@InjectMocks.

@MockBean / modern Spring test alternatives.

@SpringBootTest.

@WebMvcTest.

@DataJpaTest.

MockMvc.

Testcontainers.

Show more ▾

1. Unit Test vs Integration Test

- **Unit Test** — Tests one class/component in isolation by mocking its dependencies.
- **Integration Test** — Tests how multiple real components work together, often including Spring context, database, or external infrastructure.

Unit:

Service → Mock Repository



Integration:

Controller → Service → Repository → Database

2. JUnit 5

JUnit 5 is a Java testing framework used to write and execute automated tests using annotations such as `@Test` , `@BeforeEach` , and `@AfterEach` .

</> Java

```
@Test
void shouldCreateUser() {
    // arrange
    // act
```



```
// assert  
}
```

Common annotations:

```
@Test  
@BeforeEach  
@AfterEach  
@BeforeAll  
@AfterAll  
@ParameterizedTest
```



3. Mockito

Mockito is a mocking framework used to create and configure test doubles for dependencies in unit tests.

Example:

```
</> Java  
  
when(userRepository.findById(1L))  
    .thenReturn(Optional.of(user));
```



Then verify:

```
</> Java  
  
verify(userRepository).findById(1L);
```



4. @Mock

@Mock creates a Mockito mock object that behaves as a fake dependency during a unit test.

```
</> Java  
  
@Mock  
private UserRepository userRepository;
```



It does **not** create the real repository implementation.

5. @InjectMocks

`@InjectMocks` creates the class under test and injects the Mockito mocks into its dependencies.

</> Java



```
@Mock
private UserRepository repository;

@InjectMocks
private UserService service;
```

Conceptually:

```
UserService
  ↓
Mock UserRepository
```



6. @MockBean / Modern Spring Test Alternatives

`@MockBean` historically added a Mockito mock to the Spring `ApplicationContext` and replaced a matching bean, but in newer Spring Boot versions it has been deprecated in favor of Spring Framework's `@MockitoBean`.

Modern example:

</> Java



```
@MockitoBean
private UserService userService;
```

Use Spring's mock-bean support when you are testing with a Spring context and need to replace a real bean with a mock.

7. @SpringBootTest

@SpringBootTest loads a broad/full Spring Boot application context and is mainly used for integration-style testing.

</> Java



```
@SpringBootTest
class OrderIntegrationTest {
}
```

Conceptually:

Spring Boot Context



Controller



Service



Repository



Database



It is powerful but generally slower than focused test slices.

8. @WebMvcTest

@WebMvcTest loads only the Spring MVC/web layer and is commonly used to test controllers without loading the entire application.

</> Java



```
@WebMvcTest(UserController.class)
class UserControllerTest {
}
```

Typically:

Controller



Mock Service



It is useful for testing:

- Request mapping

- Validation
 - HTTP status
 - JSON request/response
 - Exception handling
 - Controller behavior
-

9. @DataJpaTest

@DataJpaTest loads a focused JPA test context for testing repositories and entity persistence behavior.

 `</> Java`

```
@DataJpaTest
class UserRepositoryTest {
}
```

It typically configures:


Repository
EntityManager
JPA
Database/Test database

It is useful for testing:

- Repository queries
 - Entity mappings
 - Relationships
 - JPQL/native queries
 - Persistence behavior
-

10. MockMvc

MockMvc allows you to test Spring MVC controllers by sending simulated HTTP requests without starting a real web server.

Example:

</> Java



```
mockMvc.perform(  
    get("/users/1")  
)  
    .andExpect(status().isOk());
```

You can test:

HTTP method
URL
Headers
Request body
Status code
Response JSON
Validation



11. Testcontainers

Testcontainers runs real infrastructure components such as databases, Kafka, or Redis in disposable containers during tests.

Example:

```
JUnit Test  
  ↓  
Testcontainers  
  ↓  
PostgreSQL Container  
  ↓  
Repository
```



This is useful when an embedded database does not accurately represent production behavior.

For example:

Production → PostgreSQL

Test → PostgreSQL container



instead of:

Production → PostgreSQL



Test → H2

12. How do you test REST Controllers?

I would use `@WebMvcTest` and `MockMvc`, mocking the service layer and verifying HTTP behavior.

Example:

</> Java



```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockitoBean
    UserService userService;

    @Test
    void shouldGetUser() throws Exception {

        when(userService.getUser(1L))
            .thenReturn(new UserDto(1L, "Manoj"));

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Manoj"));
    }
}
```

You should test:

Request mapping
HTTP status
Request validation
Request/response JSON
Exception handling
Authorization rules where appropriate



13. How do you test Service Classes?

I would use a pure JUnit + Mockito unit test and mock repositories/external dependencies.

</> Java



```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    UserRepository repository;

    @InjectMocks
    UserService service;

    @Test
    void shouldReturnUser() {

        User user = new User(1L, "Manoj");

        when(repository.findById(1L))
            .thenReturn(Optional.of(user));

        User result = service.getUser(1L);

        assertEquals("Manoj", result.getName());

        verify(repository).findById(1L);
    }
}
```

Test:

Business logic
Validation
Exception handling
Branching
Repository interaction
Business rules



14. How do you test Repositories?

I would use `@DataJpaTest` and test the repository against a real test database, preferably the same database technology as production when database-specific behavior matters.

Example:

`</>` Java



```
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    UserRepository repository;

    @Test
    void shouldFindByEmail() {

        User user = new User();
        user.setEmail("test@example.com");

        repository.save(user);

        Optional<User> result =
            repository.findByEmail("test@example.com");

        assertTrue(result.isPresent());
    }
}
```

For production-like database behavior:

```
@DataJpaTest
    ↓
Testcontainers
    ↓
PostgreSQL
```



15. How do you test External API Calls?

I would avoid calling the real external service in normal tests and use a mock HTTP server such as WireMock or MockWebServer to simulate the external API.

Example:

Service Under Test



Mock HTTP Server



Fake response



Test scenarios such as:

200 → Success

400 → Bad request

401 → Unauthorized

404 → Not found

500 → Server error

Timeout

Slow response

Malformed response



For a small unit test, mock the HTTP client itself; for an integration-style client test, prefer a mock HTTP server so the HTTP serialization, headers, status handling, and client configuration are actually exercised.

16. What should be mocked and what shouldn't?

Mock

Mock external dependencies whose behavior is not the subject of the unit test.

Examples:

Repository

External REST client

Kafka producer

Third-party service

Email service

Payment gateway



For a service unit test:

UserService



Mock Repository





Don't mock unnecessarily

Don't mock the class you are actually testing or simple domain objects/value objects.

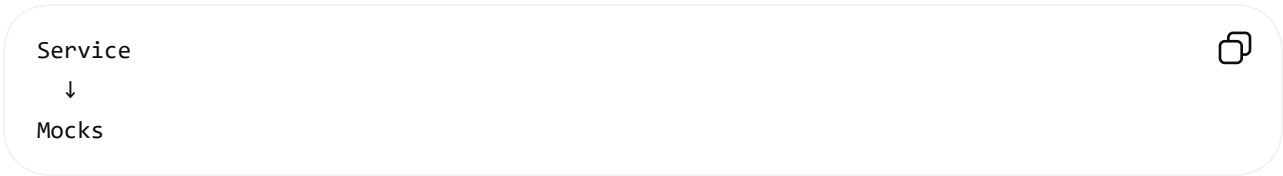
For repository integration tests, don't mock the JPA repository itself—you are specifically testing it.

For controller tests, mock the service because the controller is the subject.

17. Unit Test vs Integration Test Trade-offs

	Unit Test	Integration Test
Speed	Very fast	Slower
Scope	Small	Larger
Dependencies	Mocked	Real/partially real
Debugging	Easier	More complex
Confidence in integration	Lower	Higher
Cost	Low	Higher
Typical tool	JUnit + Mockito	Spring Boot Test/Testcontainers

Unit test

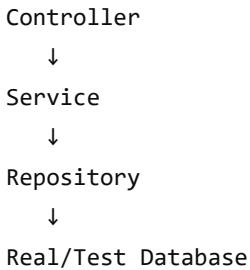


- Advantages:**
- Fast
 - Isolated
 - Easy to run frequently

- Easy to identify business-logic failures

Disadvantages:

- Can miss integration problems
- Mocks may behave differently from real dependencies

Integration test

```
graph TD; Controller --> Service; Service --> Repository; Repository --> RealTestDatabase[Real/Test Database];
```

Controller
↓
Service
↓
Repository
↓
Real/Test Database

**Advantages:**

- Tests real integration
- Detects mapping/query/configuration issues
- Higher confidence in component interaction

Disadvantages:

- Slower
- More setup
- More infrastructure
- Potentially more difficult to diagnose

Strong 5-year interview answer

"I prefer a testing pyramid: many fast unit tests for business logic, focused integration tests for database and framework integration, and a smaller number of end-to-end tests for critical user flows. I don't mock everything; I mock dependencies when testing a unit in isolation, but I use real infrastructure or Testcontainers when the integration itself is what I need to verify."



ChatGPT can make mistakes. Check important info.

